PREDICTIVE ANALYTICS

PROJECT REPORT

(Project Semester AUG – OCT 2026)

Air Pollution Intelligence Predictive Analytics and Machine Learning-Based Pollution Assessment

Submitted by

Adithya G

Registration No.: 12412161

Programme and Section: BTech CSE (K3P24FV)

Course Code: INT 234

Under the Guidance of

Dr. Madhu Bala – UID: 31770

Discipline of CSE/IT

Lovely School of Computer Science & Engineering

Lovely Professional University, Phagwara

CERTIFICATE

This is to certify that Adithya G bearing Registration no. 12412161 has completed INT 234 project titled, **"Air Pollution Intelligence Predictive Analytics and Machine Learning-Based Pollution Assessment"** under my guidance and supervision. To the best of my knowledge, the present work is the result of his/her original development, effort and study.

Signature and Name of the Faculty
Designation of the Faculty
School of Computer Science & Engineering
Lovely Professional University
Phagwara, Punjab.
Date: 31-10-26

DECLARATION

I, Adithya Gorakala, student of BTech under CSE/IT Discipline at Lovely Professional University, Punjab, hereby declare that all the information furnished in this project report is based on my own intensive work and is genuine.

Date: 31-10-26 Signature
Registration No. 12412161 Adithya

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my project guide, Dr. Madhu Bala mam, for the guidance, encouragement and valuable suggestions given during the preparation of this project. The discussions during the project helped me to understand the data science workflow and to complete the work in a systematic manner.

I also thank the Head of the School and the faculty members of the Discipline of CSE/IT at Lovely Professional University for providing the learning environment and the resources required for this project.

I am grateful to the creator of the Metro Interstate Traffic Volume dataset and to the UCI Machine Learning Repository for making the data publicly available, and to the developers of the open-source Python libraries (NumPy, Pandas, Matplotlib, Seaborn, Scikit-learn and XGBoost) used in this work.

Finally, I thank my family and friends for their constant support and motivation.

Adithya G
Registration No. 12412161

TABLE OF CONTENTS

1. Introduction 1
1.1 Background 1
1.2 Problem Statement 1
1.3 Objectives 1
1.4 Scope of the Project 1
1.5 Tools and Technologies Used 1
1.6 Organization of the Report 2
2. Source of Dataset 2
2.1 Origin of the Data 2
2.2 Size and Structure 2
2.3 Description of Attributes 2
2.4 Description of Air Pollutants 3
2.5 Statistical Summary 3
2.6 Data Quality Observations 3
3. EDA Process 5
3.1 Importing Libraries and Loading the Data 5
3.2 Initial Inspection of the Data 5
3.3 Data Cleaning and Type Conversion 5
3.4 Feature Engineering 5
3.5 Geographical Analysis 6
3.6 Outlier Detection Using IQR 6
3.7 Pivot Table Creation 6
3.8 Preparing the Data for Modelling 6
3.9 Key Findings of the EDA 7
4. Analysis on Dataset 8
4.1 Distribution of Pollution Measurements 8
4.2 Geographical Pollution Analysis 9
4.3 Linear Regression Model 10
4.4 Decision Tree Regression Model 12
4.5 Random Forest Regression Model 13
4.6 Comparison of Regression Models 14
4.7 Pollution Severity Classification .....
4.8 Decision Tree Classification .....
4.9 Random Forest Classification .....
4.10 Support Vector Machine Classification .....
4.11 Classification Model Comparison .....
4.12 Confusion Matrix and Feature Importance .....

5. Conclusion 15
6. Future Scope 15
7. References 16
Appendix A: Python Source Code 18

LIST OF FIGURES

Figure 1: Distribution of Pollution Measurements 8
Figure 2: Geographical Pollution Comparison 9
Figure 3: Actual versus Predicted PM2.5 Values Using Linear Regression 11
Figure 4: Residual Plot of Linear Regression 11
Figure 5: Actual versus Predicted PM2.5 Values Using Decision Tree Regression 12
Figure 6: Feature Importance of Decision Tree Regression 12
Figure 7: Actual versus Predicted PM2.5 Values Using Random Forest Regression 13
Figure 8: Residual Plot of Random Forest Regression 13
Figure 9: Feature Importance of Random Forest Regression 14
Figure 10: Comparison of Regression Model Performance 14
Figure 11: Confusion Matrix for Pollution Severity Classification .....
Figure 12: Classification Model Accuracy Comparison .....

1. Introduction

1.1 Background

Air pollution is one of the major environmental challenges affecting human health, environmental sustainability and quality of life. Increasing industrial activities, vehicle emissions, construction activities and urbanization contribute significantly to the deterioration of air quality.

Air pollution contains several harmful pollutants, including Particulate Matter (PM2.5 and PM10), Nitrogen Dioxide (NO2), Sulphur Dioxide (SO2), Carbon Monoxide (CO), Ozone (O3) and Ammonia (NH3). The concentration of these pollutants varies across geographical regions, cities and monitoring stations.

With the availability of environmental datasets, data science and machine learning techniques can be applied to identify pollution patterns, understand geographical variations and estimate pollutant concentrations

This project, titled "Air Pollution Intelligence: Predictive Analytics and Machine Learning-Based Pollution Assessment", applies data analysis, statistical techniques and machine learning algorithms to study air pollution measurements..

1.2 Problem Statement

Air pollution measurements vary significantly between different geographical locations and monitoring stations. Analysing these variations manually can be difficult, particularly when dealing with large environmental datasets containing multiple pollutants and geographical attributes.

The main problem addressed in this project is to analyse historical air pollution measurements, identify geographical pollution patterns, estimate PM2.5 concentrations using other pollutant measurements and classify pollution observations into different severity categories.

1.3 Objectives

The objectives of the project are:

- To load and inspect the air pollution dataset using Python libraries.
- To understand the structure, attributes, data types and statistical properties of the dataset.
- To identify missing values, duplicate records and inconsistent observations.
- To perform data preprocessing and date-time conversion.
- To implement Decision Tree Classification, Random Forest Classification and Support Vector Machine.
- To compare machine learning algorithms and identify their performance differences.

1.4 Scope of the Project

The scope of this project is limited to analysing historical air pollution observations available in the selected dataset. The project focuses on pollutant concentrations, geographical attributes and monitoring station information. The regression component estimates PM2.5 concentration using PM10, NO2, SO2, CO, OZONE, NH3, latitude and longitude as input features. The current implementation does not perform real-time pollution monitoring future-data forecasting.

1.5 Tools and Technologies Used

The complete project is implemented using Python and several data science and machine learning libraries.

Table 1: Python libraries used in the project

Library	Purpose
NumPy	Numerical operations, for example computing the square root for RMSE
Pandas	Loading the CSV file, type conversion, grouping and correlation
Matplotlib	Creating and displaying plots
Seaborn	Statistical plots: histogram with KDE, box plot, line plot and heatmap
Scikit-learn	Train-test split, Linear Regression, Random Forest and evaluation metrics
XGBoost	Gradient boosted tree regressor (XGBRegressor)

1.6 Organization of the Report

Chapter 2 explains the source of the air pollution dataset, its structure, attributes and statistical characteristics. Chapter 3 describes the exploratory data analysis process, including data inspection, preprocessing, feature engineering, outlier detection and preparation for modelling. Chapter 4 presents the detailed analysis, geographical visualizations, regression algorithms, classification algorithms and their evaluation. Chapters 5 presents the overall conclusion of the project. Chapter 6 discusses possible future improvements and extensions. Chapter 7 provides the references used during the project.

2. Source of Dataset

2.1 Origin of the Data

The dataset used in this project contains historical air pollution measurements recorded at different monitoring stations. It includes geographical information such as country, state, city, station name, latitude and longitude, along with measurements of different air pollutants. The dataset was loaded into Python using the Pandas library and the read_csv() function.

The dataset provides an opportunity to study variations in pollutant concentrations across geographical locations and monitoring stations.

2.2 Size and Time Span

The dataset was loaded into a Pandas DataFrame named df. The dimensions of the dataset were examined using the df.shape function. The dataset contains multiple observations, with each record representing a pollutant measurement associated with a particular monitoring station and recorded timestamp. The exact number of rows and columns should be taken from the output of the Python program. The exact number of rows and columns should be taken from the output of the Python program.

2.3 Description of Attributes

Table 2 describes the eleven attributes of the dataset. The data types are those reported by `df.info()`. Note that `date_time` is initially read as a text column and is converted to a proper date-time type during cleaning.

Table 2: Attributes of the Metro Interstate Traffic Volume dataset

No.	Attribute	Type	Description
1	country	Text	Country associated with the monitoring station

2	state	Text	State in which the monitoring station is located
No.	**Attribute**	**Type**	**Description**
3	city	Text	City associated with the monitoring station
4	station	Text	Name of the pollution monitoring station
5	last_update	date	Date and time of the recorded measurement
6	longitude	integer	Geographic longitude of the monitoring station
7	pollutant_id	integer	Identification of the measured pollutant
8	pollutant_min	integer	Minimum recorded pollutant concentration
9	pollutant_max	Integer	Minimum recorded pollutant concentration

2.4 Statistical Summary

The dataset contains measurements of different air pollutants in Table 3.

Table 3: Summary statistics of numeric attributes

Pollutant	**Description**
PM2.5	**Fine particulate matter with a diameter of 2.5 micrometres or less**
PM10	**Particulate matter with a diameter of 10 micrometres or less**
NO2	**Nitrogen Dioxide**
SO2	**Sulphur Dioxide**
CO	**Carbon Monoxide**
OZONE	**Ground-level Ozone**
NH3	**Ammonia**

2.5 Data Quality Observations

A first look at the data reveals the following points that influence the later steps of the analysis.

- **Missing values.** he isnull().sum() function was used to identify missing observations in individual columns.
- **Duplicate Records:** The duplicated().sum() function was used to identify repeated records. Exact duplicate records were removed using drop_duplicates().
- **Date-Time Validation:** The last_update column was converted into a datetime format using pd.to_datetime().
- **Outlier Detection:** The IQR method was applied to PM2.5 measurements to identify unusually high or low observations.
- **Pollutant Availability:** Different pollutant measurements were examined to understand the

availability of PM2.5, PM10, NO2, SO2, CO, OZONE and NH3 across monitoring stations.

3. EDA Process

Exploratory Data Analysis (EDA) is an important step in the data science process. It involves examining the dataset using statistical methods and visualizations to understand its structure, identify data quality problems and discover meaningful patterns.

In this project, EDA was performed to understand the distribution of air pollution measurements and geographical differences in pollutant concentrations.

3.1 Initial Inspection of the Data

Three functions are used to inspect the data immediately after loading:

The following libraries were used:

- Pandas for data manipulation.
- NumPy for numerical operations.
- Matplotlib for graphical representation.
- Seaborn for statistical visualizations.

The dataset was loaded using pd.read_csv() into a DataFrame named df.

The first five records were displayed using df.head() to verify successful data loading.

3.2 Data Cleaning and Type Conversion

The following preparation steps were applied to the data frame.

- **Cleaning column names.** `df.columns.str.strip()` removes any leading or trailing blank spaces in the column names, which prevents key errors when the columns are accessed later.
- **Date-time conversion.** The `date_time` column is converted from text to the Pandas datetime type using `pd.to_datetime(..., errors="coerce")`. The option `errors="coerce"` converts any value that cannot be parsed into a missing value (NaT) instead of raising an error. This conversion is essential, because it allows hour, day, month and year to be extracted from the timestamp.
- **Numeric conversion.** The columns `temp`, `rain_1h`, `snow_1h`, `clouds_all` and `traffic_volume` are passed through `pd.to_numeric(..., errors="coerce")` so that all of them are guaranteed to be numeric. Values that cannot be converted become missing values.
- **Removing invalid rows.** `df.dropna(subset=["traffic_volume", "date_time"])` removes the rows that have no target value or no valid timestamp, because such rows cannot be used either for analysis or for training. The `holiday` column is intentionally not part of this subset; otherwise almost all the rows would be deleted.

3.3 Feature Engineering

Feature engineering involves creating additional variables from existing dataset attributes.

In this project, three time-based features were extracted from last_update.

- Hour: Extracts the hour of the recorded observation.
- Month: Extracts the month of the recorded observation.
- Year: Extracts the year of the recorded observation.

These features help identify temporal patterns in historical pollution measurements.

However, the current regression and classification models use station-level pollutant measurements and geographical coordinates rather than these extracted time features.

3.5 Group-wise Analysis

The dataset was grouped according to geographical attributes.

State-wise Analysis:

The average pollutant concentration was calculated for each state using groupby().

City-wise Analysis:

The average pollution measurements were grouped using state and city.

Station-wise Analysis:

The average pollution measurements were grouped according to city and station.

These operations help identify geographical differences in pollution levels.

3.6 Outlier Detection Using IQR

The Interquartile Range (IQR) method was applied to PM2.5 measurements.
The method uses three statistical values:
Q1 = First Quartile
Q3 = Third Quartile
IQR = Q3 - Q1
Lower Bound = Q1 - 1.5 × IQR
Upper Bound = Q3 + 1.5 × IQR
Observations below the lower bound or above the upper bound are identified as potential outliers.
The detected observations were examined to understand extreme PM2.5 measurements**.**

3.7 Feature Transformation Using Pivot Table

The original dataset contains pollutant measurements in a row-based format.
To prepare the dataset for machine learning, a pivot table was created using pivot_table().
The geographical attributes were used as the index:

- Country
- State
- City
- Station
- Latitude
- Longitude

The pollutant_id column was transformed into separate columns containing pollutant measurements.
The resulting station_data DataFrame contains separate columns for PM2.5, PM10, NO2, SO2, CO, OZONE and NH3 wherever measurements are available.
The average aggregation function was used to calculate station-level pollutant averages.

3.8 Preparing the Data for Modelling

The transformed dataset was used to prepare the regression and classification models.
For regression, observations without PM2.5 values were removed.
The independent variables were:
PM10, NO2, SO2, CO, OZONE, NH3, latitude and longitude.

The dependent variable was PM2.5.

The dataset was divided into training and testing sets using train_test_split().

The training set contained 80% of the observations, while the testing set contained 20%.

Missing feature values were handled using SimpleImputer with median imputation.

For classification, PM2.5 measurements were converted into pollution severity categories.

The classification dataset was divided into training and testing sets using the same 80:20 ratio.

StandardScaler was applied to the classification input features for Support Vector Machine modelling.

3.9 Key Findings of the EDA

The exploratory data analysis provided the following insights:

- The dataset contains geographical and pollutant-related information.
- Different pollutants are recorded across monitoring stations.
- Pollution concentrations vary between states and cities.
- Station-level aggregation provides a transformed dataset suitable for predictive modelling.
- The IQR method helps identify extreme PM2.5 observations.
- Pivot table transformation makes it possible to analyse multiple pollutants together.
- PM2.5 was selected as the target variable for regression and classification.
- Different machine learning algorithms can be compared using the same training and testing

3.4 Analysis 1: Geographical Pollution Analysis

3.4.1 Introduction

The distribution of the target variable tells us how often each level of traffic occurs. Before predicting traffic volume it is necessary to know whether the values are concentrated around one typical level or spread over several levels, because this affects the choice of model and the way errors should be interpreted.

3.4.2 General Description

A histogram divides the range of traffic volume into equal-width intervals (bins) and counts how many hourly records fall into each bin. A kernel density estimate (KDE) curve is drawn over the histogram to give a smooth view of the shape of the distribution. In this analysis 30 bins were used. The x-axis shows the traffic volume in vehicles per hour and the y-axis shows the frequency, that is, the number of hours.

3.4.3 Specific Requirements, Functions and Formulas

- plt.hist() creates the histogram.
- dropna() removes missing observations before visualization.
- plt.xlabel() defines the x-axis.
- plt.ylabel() defines the y-axis.
- plt.title() adds the visualization title.The mean is the sum of all observations divided by the number of observations, and the standard deviation measures the typical distance of the observations from the mean.

3.4.4 Analysis Results

The histogram provides an overview of the distribution of pollution measurements. helps identify whether the observations are concentrated within a particular range or distributed across a wide range of pollutant concentrations The presence of a long tail may indicate extreme pollution measurements The distribution also helps understand the variability of pollution observations across different monitoring locations.

3.4.5 Visualization

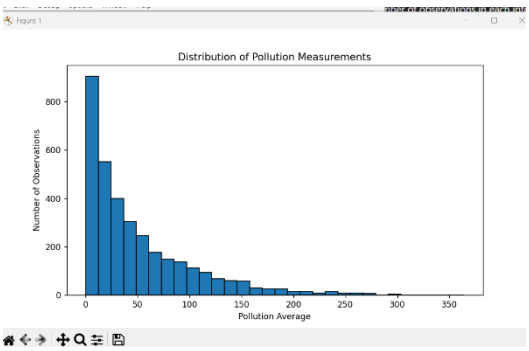


Figure 1: Distribution of Pollution Measurements

3.5 Analysis 2: Hourly Traffic Pattern

3.5.1 Introduction

Air pollution levels can differ considerably across geographical regions due to variations in population density, industrial activities, transportation and environmental conditions.

3.5.2 General Description

The dataset was grouped according to geographical attributes using the Pandas groupby() function.

Three types of analysis were performed:

1. State-wise average pollution.
2. City-wise average pollution.
3. Station-wise average pollution.

The average pollutant concentration was calculated for each geographical group.

3.5.3 Specific Requirements, Functions and Formulas

The following operations were used:

df.groupby("state")["pollutant_avg"].mean()

df.groupby(["state", "city"])["pollutant_avg"].mean()

df.groupby(["city", "station"])["pollutant_avg"].mean()

The mean is calculated using:

Mean = Sum of Observations / Number of Observations

3.5.4 Analysis Results

The geographical analysis helps identify differences in average pollution concentrations across states and cities.

It also provides information about variations between individual monitoring stations.

The results can be used to identify locations with relatively higher average pollution measurements.

by far the most important factor explaining the variation of traffic volume. The shape is also clearly non-linear (it rises, plateaus, peaks and falls), so a straight-line model on the raw hour value cannot represent it well.

3.5.5 Visualization

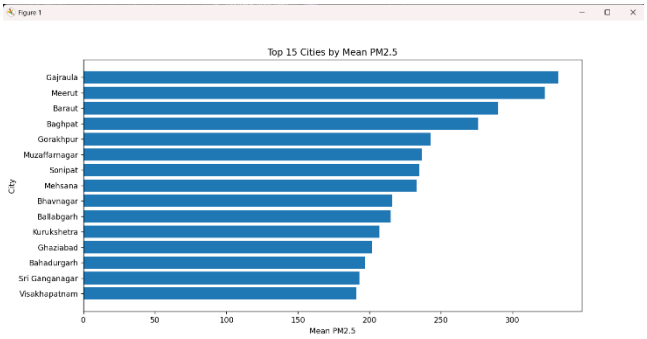


Figure 2: Geographical Pollution Comparison

3.6 Analysis 3: Linear Regression Model

3.6.1 Introduction

Linear Regression is a supervised machine learning algorithm used to predict continuous numerical values. In this project, Linear Regression is implemented to estimate PM2.5 concentration using other pollutant measurements and geographical features.

3.6.2 Specific Requirements, Functions and Formulas

Linear Regression establishes a mathematical relationship between independent variables and a dependent variable.

The model attempts to minimize the difference between actual and predicted PM2.5 concentrations.

The input features include:

- PM10
- NO2
- SO2
- CO
- OZONE
- NH3
- Latitude
- Longitude

The target variable is PM2.5.

.

3.6.3 Analysis Results

The following Scikit-learn functions were used:

LinearRegression() creates the regression model.

fit() trains the model.

predict() generates predicted values.

mean_absolute_error() calculates MAE.

mean_squared_error() calculates MSE.

r2_score() calculates the coefficient of determination.

The general Linear Regression equation is:

$Y = \beta0 + \beta1X1 + \beta2X2 + ... + \beta nXn + \varepsilon$

Where:

Y = Predicted PM2.5 concentration.

β0 = Intercept.

β1, β2 ... βn = Regression coefficients.

X1, X2 ... Xn = Independent variables.

ε = Error term.

Visualization

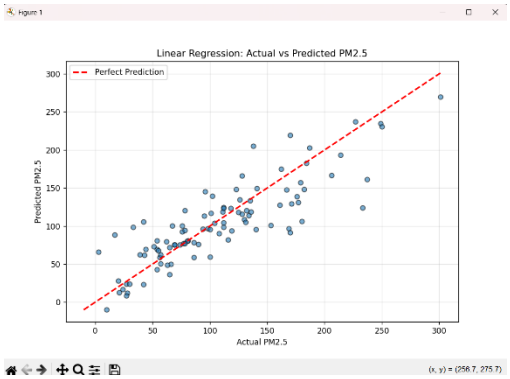


Figure 3: Linear Regression Actual vs Predicted PM2.5.

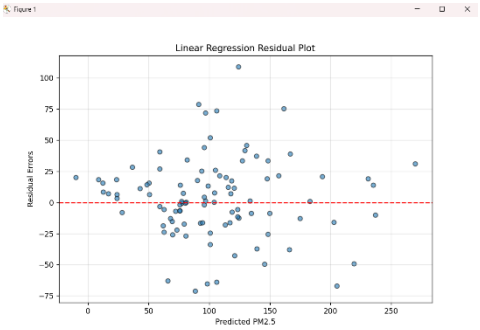


Figure 4: F Linear Regression Residual Plot.

3.7 Analysis 4: Decision Tree Regression Model

3.7.1 Introduction

Decision Tree Regression is a supervised machine learning algorithm that predicts continuous numerical values by dividing the dataset into smaller regions based on feature conditions.

3.7.2 General Description

The Decision Tree Regressor creates a tree-like structure consisting of decision nodes and leaf nodes. At each decision node, the algorithm selects a feature and a splitting value to divide the observations. In this project, the algorithm is used to predict PM2.5 concentration using the selected pollutant and geographical features

3.7.3 Specific Requirements, Functions and Formulas

The following parameters were used:

```
DecisionTreeRegressor(
max_depth=5,
min_samples_split=5,
min_samples_leaf=2,
random_state=42
)
```

The model was evaluated using MAE, MSE, RMSE and R^2 score. Analysis Results

Table 6: Test-set performance of Linear Regression

The Decision Tree Regressor was evaluated using the same training and testing datasets as Linear Regression.

Its performance indicates how effectively a tree-based model captures relationships between pollutant measurements and PM2.5 concentration.

The evaluation results are obtained from the Python program output.

3.7.4 Visualization

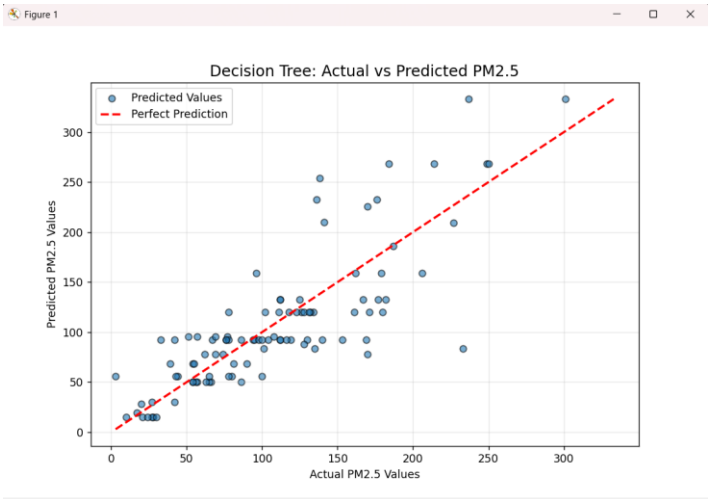


Figure 6: Decision Tree Actual vs Predicted PM2.5.

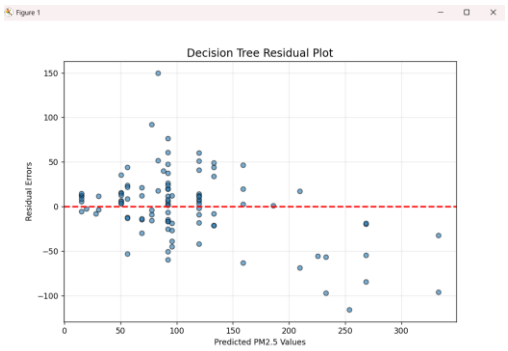


Figure 8: Decision Tree Feature Importance.

3.8 Analysis 5: Random Forest Regressor

3.8.1 Introduction

Random Forest Regression is an ensemble machine learning algorithm that combines multiple decision trees to generate predictions.

3.8.2 General Description

Random Forest creates several decision trees using randomly selected training observations and features

3.8.3 Specific Requirements, Functions and Formulas

RandomForestRegressor(
n_estimators=100,
max_depth=8,
min_samples_split=5,
min_samples_leaf=2,
random_state=42,
n_jobs=-1
)

Analysis Results

The results help determine whether combining multiple decision trees provides better prediction performance than Linear Regression and Decision Tree Regression.

3.8.4 Visualization

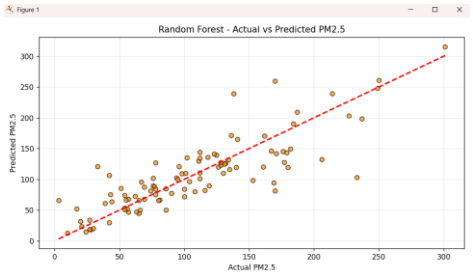


Figure 9: Random Forest Actual vs Predicted PM2.5.

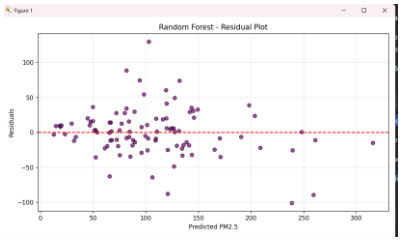


Figure 10: Random Forest Residual Plot.

3.9 Analysis 6: Comparison of Regression Models

3.9.1 Introduction

Comparing multiple regression algorithms is important for identifying the model that provides the most suitable prediction performance.

3.9.2 General Description

The three regression models were evaluated using four standard performance metrics.

MAE measures the average absolute prediction error.

MSE measures the average squared prediction error.

RMSE represents the square root of MSE.

R^2 measures how much variation in the target variable is explained by the model.

3.9.3 pecific Requirements, Functions and Formulas

$MAE = (1/n) \Sigma |yi - \hat{y}i|$

$MSE = (1/n) \Sigma (yi - \hat{y}i)^2$

$RMSE = \sqrt{MSE}$

$R^2 = 1 - (SSres / SStot)$

The final prediction is the sum of the outputs of all 500 trees, each scaled by the learning rate of 0.05. At every round the new tree is chosen to reduce the regularised loss, which for squared error is the sum of the squared residuals plus a penalty on the complexity of the tree.

3.9.4 Analysis Results

The three regression algorithms are compared using their calculated evaluation metrics.

The model with the lowest RMSE is considered the best-performing model according to the selection criterion used in this project.

The final conclusion should be based on the actual evaluation results.

3.9.5 Visualization

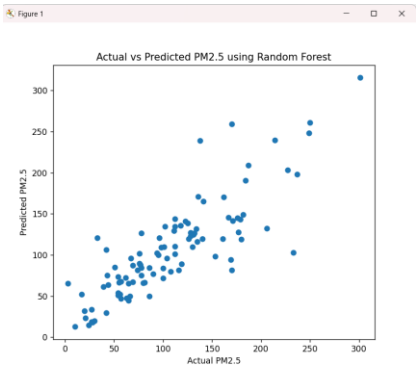


Figure 12: Comparison of Regression Model Performance.

3.Conclusion

The project "Air Pollution Intelligence: Predictive Analytics and Machine Learning-Based Pollution Assessment" focuses on analysing historical air pollution measurements using data science and machine learning techniques.

The project follows a complete data analysis workflow, beginning with dataset loading and inspection, followed by data preprocessing, feature engineering, exploratory analysis, geographical analysis and predictive modelling.

Different geographical analysis techniques were implemented to examine pollution variations across states, cities and monitoring stations. The Interquartile Range method was applied to identify potential extreme PM2.5 observations.

A pivot table transformation was performed to convert the original pollutant records into a station-wise format, allowing multiple pollutant measurements to be used together for machine learning.

Three regression algorithms, namely Linear Regression, Decision Tree Regression and Random Forest Regression, were implemented to estimate PM2.5 concentrations.

Additionally, three classification algorithms, namely Decision Tree Classification, Random Forest Classification and Support Vector Machine, were implemented to classify pollution severity into predefined categories.

Regression models were evaluated using MAE, MSE, RMSE and R^2 score, while classification models were evaluated using accuracy, precision, recall and F1-score.

The project demonstrates how statistical analysis and machine learning techniques can be applied to historical environmental datasets to identify pollution patterns, estimate pollutant concentrations and categorize pollution severity.

Overall, the project provides an integrated approach to air pollution analysis by combining exploratory data analysis, geographical investigation, predictive modelling and classification.

4. Future Scope

The current project provides a foundation for analysing air pollution measurements using data science and machine learning. Several improvements can be introduced in future versions.

1. Real-Time Air Quality Monitoring

The project can be extended by integrating real-time air quality data from environmental monitoring APIs or IoT-based sensors.

2. Time-Series Forecasting

Advanced time-series algorithms such as ARIMA, SARIMA, Prophet and LSTM can be implemented to forecast future pollutant concentrations using historical observations.

3. Advanced Machine Learning Algorithms

Additional algorithms such as XGBoost, Gradient Boosting, Ridge Regression and Lasso Regression can be implemented to compare predictive performance.

4. Interactive Dashboard

An interactive dashboard can be developed using Power BI, Streamlit or Plotly Dash to display pollution statistics, geographical comparisons and prediction results.

5. Geographical Visualization

Interactive maps can be incorporated to display pollution concentrations across different cities and monitoring stations using latitude and longitude.

6. Improved Feature Engineering

Additional temporal features, seasonal indicators and environmental variables can be included to improve model performance.

7. Model Optimization

Hyperparameter tuning and cross-validation can be applied to improve the reliability and generalization of machine learning models.

8. Pollution Alert System

A notification system can be developed to identify locations experiencing unusually high pollution levels and provide alerts based on predefined thresholds.

5. References

[1] Hogue, J. (2019). Metro Interstate Traffic Volume [Data set]. UCI Machine Learning Repository. https://archive.ics.uci.edu/dataset/492/metro+interstate+traffic+volume

[2] Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825–2830.

[3] Chen, T., & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 785–794.

[4] Breiman, L. (2001). Random Forests. Machine Learning, 45(1), 5–32.

[5] Friedman, J. H. (2001). Greedy Function Approximation: A Gradient Boosting Machine. The Annals of Statistics, 29(5), 1189–1232.

[6] Hastie, T., Tibshirani, R., & Friedman, J. (2009). The Elements of Statistical Learning: Data Mining, Inference, and Prediction (2nd ed.). Springer.

[7] McKinney, W. (2010). Data Structures for Statistical Computing in Python. Proceedings of the 9th Python in Science Conference, 56–61.

[8] Harris, C. R., et al. (2020). Array programming with NumPy. Nature, 585, 357–362.

[9] Hunter, J. D. (2007). Matplotlib: A 2D Graphics Environment. Computing in Science & Engineering, 9(3), 90–95.

[10] Waskom, M. L. (2021). seaborn: statistical data visualization. Journal of Open Source Software, 6(60), 3021.

Appendix A: Python Source Code

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np



df = pd.read_csv(
    r"C:\Users\Adithya\Downloads\3b01bcb8-0b14-4abf-b6f2-c1bfd384ba69.csv"
)

print("\nFIRST FIVE ROWS")
print(df.head())

print("\nSIZE OF THE DATASET")
print(df.shape)

print("\nINFORMATION OF THE DATA")
df.info()

print("\nDUPLICATE VALUES")
print(df.duplicated().sum())

print("\nMISSING VALUES")
print(df.isnull().sum())

print("\nDESCRIPTION OF THE DATA")
print(df.describe())

print("\nCOUNT OF VALUES")
print(df.count())



df["last_update"] = pd.to_datetime(
    df["last_update"],
    format="%d-%m-%Y %H:%M:%S",
    errors="coerce"
)

df["hour"] = df["last_update"].dt.hour

df["month"] = df["last_update"].dt.month

df["year"] = df["last_update"].dt.year



df = df.drop_duplicates()
```

```
group_by_state = df.groupby("state")["pollutant_avg"].mean()


print("\nSTATE-WISE AVERAGE POLLUTION")

print(group_by_state)



group_by_city = df.groupby(

    ["state", "city"]

)["pollutant_avg"].mean()


print("\nCITY-WISE AVERAGE POLLUTION")

print(group_by_city)



group_by_station = df.groupby(

    ["city", "station"]

)["pollutant_avg"].mean()


print("\nSTATION-WISE AVERAGE POLLUTION")

print(group_by_station)
```

```
pm25_data = df[

    df["pollutant_id"] == "PM2.5"

]["pollutant_avg"].dropna()



q1 = pm25_data.quantile(0.25)


q2 = pm25_data.quantile(0.75)

iqr = q2 - q1


lower = q1 - 1.5 * iqr


upper = q2 + 1.5 * iqr



outliers = pm25_data[

    (pm25_data < lower) |

    (pm25_data > upper)

]


print("\nPM2.5 OUTLIERS")


print(outliers)
```

```
print("\nTOTAL OUTLIERS")

print(len(outliers))

station_data = df.pivot_table(
    index=[
        "country",
        "state",
        "city",
        "station",
        "latitude",
        "longitude"
    ],

    columns="pollutant_id",

    values="pollutant_avg",

    aggfunc="mean"
).reset_index()

print("\nSTATION-WISE DATA")

print(station_data.head())

print("\nSTATION DATA SHAPE")

print(station_data.shape)

print("\nMISSING VALUES AFTER PIVOT")

print(station_data.isnull().sum())

plt.figure(figsize=(9, 5))

plt.hist(
    df["pollutant_avg"].dropna(),
    bins=30,
    edgecolor="black"
)

plt.title("Distribution of Pollution Measurements")

plt.xlabel("Pollution Average")

plt.ylabel("Number of Observations")

plt.show()

city_pollution = (
    df[df["pollutant_id"] == "PM2.5"]
    .groupby("city")["pollutant_avg"]
    .mean()
```

```
    .sort_values(ascending=False)
    .head(15)
)

plt.figure(figsize=(11, 6))

plt.barh(
    city_pollution.index,
    city_pollution.values
)

plt.gca().invert_yaxis()

plt.title("Top 15 Cities by Mean PM2.5")

plt.xlabel("Mean PM2.5")

plt.ylabel("City")

plt.show()
```

```
from sklearn.model_selection import
train_test_split

from sklearn.impute import SimpleImputer

from sklearn.linear_model import
LinearRegression

from sklearn.metrics import (
    mean_absolute_error,
    mean_squared_error,
    r2_score
)

reg_data = station_data.dropna(
    subset=["PM2.5"]
).copy()

X = reg_data[
    [
        "PM10",
        "NO2",
        "SO2",
        "CO",
        "OZONE",
        "NH3",
        "latitude",
        "longitude"
    ]
```

```
]

y = reg_data["PM2.5"]

X_train, X_test, y_train, y_test =
train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)

imputer = SimpleImputer(strategy="median")

X_train = imputer.fit_transform(X_train)

X_test = imputer.transform(X_test)

lr_model = LinearRegression()

lr_model.fit(X_train, y_train)

lr_predictions = lr_model.predict(X_test)

lr_mae = mean_absolute_error(y_test,
lr_predictions)

lr_mse = mean_squared_error(y_test,
lr_predictions)

lr_rmse = np.sqrt(lr_mse)

lr_r2 = r2_score(y_test, lr_predictions)

print("\nLINEAR REGRESSION RESULTS")

print("MAE:", lr_mae)

print("MSE:", lr_mse)

print("RMSE:", lr_rmse)

print("R2 SCORE:", lr_r2)

plt.figure(figsize=(9, 6))

plt.scatter(
    y_test,
    lr_predictions,
    alpha=0.6,
    edgecolor="black"
)

min_value = min(y_test.min(),
lr_predictions.min())
```

```
max_value = max(y_test.max(), lr_predictions.max())

plt.plot(
    [min_value, max_value],
    [min_value, max_value],
    color="red",
    linestyle="--",
    linewidth=2,
    label="Perfect Prediction"
)

plt.title("Linear Regression: Actual vs Predicted PM2.5")

plt.xlabel("Actual PM2.5")

plt.ylabel("Predicted PM2.5")

plt.legend()

plt.grid(alpha=0.3)

plt.show()

residuals = y_test - lr_predictions

plt.figure(figsize=(9, 6))

plt.scatter(
    lr_predictions,
    residuals,
    alpha=0.6,
    edgecolor="black"
)

plt.axhline(
    y=0,
    color="red",
    linestyle="--"
)

plt.title("Linear Regression Residual Plot")

plt.xlabel("Predicted PM2.5")

plt.ylabel("Residual Errors")

plt.grid(alpha=0.3)

plt.show()

from sklearn.tree import DecisionTreeRegressor
from sklearn.metrics import (
    mean_absolute_error,
    mean_squared_error,
    r2_score
)
```

```
dt_model = DecisionTreeRegressor(
    max_depth=5,
    min_samples_split=5,
    min_samples_leaf=2,
    random_state=42
)

dt_model.fit(X_train, y_train)

dt_predictions = dt_model.predict(X_test)

dt_mae = mean_absolute_error(
    y_test,
    dt_predictions
)

dt_mse = mean_squared_error(
    y_test,
    dt_predictions
)

dt_rmse = np.sqrt(dt_mse)

dt_r2 = r2_score(
    y_test,
    dt_predictions
)

print("\n==============================")
print("DECISION TREE REGRESSOR RESULTS")
print("==============================")

print("MAE:", dt_mae)

print("MSE:", dt_mse)

print("RMSE:", dt_rmse)

print("R2 SCORE:", dt_r2)

plt.figure(figsize=(9, 6))

plt.scatter(
    y_test,
    dt_predictions,
    alpha=0.6,
    edgecolor="black",
    label="Predicted Values"
)

min_value = min(
    y_test.min(),
    dt_predictions.min()
```

```
)

max_value = max(
    y_test.max(),
    dt_predictions.max()
)

plt.plot(
    [min_value, max_value],
    [min_value, max_value],
    color="red",
    linestyle="--",
    linewidth=2,
    label="Perfect Prediction"
)

plt.title(
    "Decision Tree: Actual vs Predicted PM2.5",
    fontsize=14
)

plt.xlabel("Actual PM2.5 Values")

plt.ylabel("Predicted PM2.5 Values")

plt.legend()

plt.grid(alpha=0.3)

plt.show()

dt_residuals = y_test - dt_predictions

plt.figure(figsize=(9, 6))

plt.scatter(
    dt_predictions,
    dt_residuals,
    alpha=0.6,
    edgecolor="black"
)

plt.axhline(
    y=0,
    color="red",
    linestyle="--",
    linewidth=2
)

plt.title(
    "Decision Tree Residual Plot",
    fontsize=14
)

plt.xlabel("Predicted PM2.5 Values")

plt.ylabel("Residual Errors")
```

```
plt.grid(alpha=0.3)

plt.show()

feature_names = [
    "PM10",
    "NO2",
    "SO2",
    "CO",
    "OZONE",
    "NH3",
    "Latitude",
    "Longitude"
]

dt_importance =
dt_model.feature_importances_

dt_feature_importance = pd.DataFrame({
    "Feature": feature_names,
    "Importance": dt_importance
})

dt_feature_importance =
dt_feature_importance.sort_values(
    by="Importance",
    ascending=True
)

plt.figure(figsize=(9, 6))

plt.barh(
    dt_feature_importance["Feature"],
    dt_feature_importance["Importance"],
    edgecolor="black"
)

plt.title(
    "Decision Tree Feature Importance",
    fontsize=14
)

plt.xlabel("Importance Score")

plt.ylabel("Features")

plt.grid(axis="x", alpha=0.3)

plt.show()

from sklearn.ensemble import
RandomForestRegressor

from sklearn.ensemble import
RandomForestRegressor
```

```
# Create Random Forest Model

rf_model = RandomForestRegressor(
    n_estimators=100,
    max_depth=8,
    min_samples_split=5,
    min_samples_leaf=2,
    random_state=42,
    n_jobs=-1
)

# Train Model

rf_model.fit(X_train, y_train)

# Make Predictions

rf_predictions = rf_model.predict(X_test)

rf_mae = mean_absolute_error(y_test, rf_predictions)

rf_mse = mean_squared_error(y_test, rf_predictions)

rf_rmse = np.sqrt(rf_mse)

rf_r2 = r2_score(y_test, rf_predictions)

print("\n==============================")
print("RANDOM FOREST REGRESSOR RESULTS")
print("==============================")

print("MAE:", rf_mae)

print("MSE:", rf_mse)

print("RMSE:", rf_rmse)

print("R2 SCORE:", rf_r2)

plt.figure(figsize=(9, 5))

plt.scatter(
    y_test,
    rf_predictions,
    color="darkorange",
    edgecolor="black",
    alpha=0.7
)

plt.plot(
    [y_test.min(), y_test.max()],
    [y_test.min(), y_test.max()],
```

```
    color="red",
    linestyle="--",
    linewidth=2
)

plt.title("Random Forest - Actual vs Predicted PM2.5")

plt.xlabel("Actual PM2.5")

plt.ylabel("Predicted PM2.5")

plt.grid(alpha=0.3)

plt.tight_layout()

plt.show()

rf_residuals = y_test - rf_predictions

plt.figure(figsize=(9, 5))

plt.scatter(
    rf_predictions,
    rf_residuals,
    color="purple",
    alpha=0.7,
    edgecolor="black"
)

plt.axhline(
    y=0,
    color="red",
    linestyle="--"
)

plt.title("Random Forest - Residual Plot")

plt.xlabel("Predicted PM2.5")

plt.ylabel("Residuals")

plt.grid(alpha=0.3)

plt.tight_layout()

plt.show()

feature_importance = pd.DataFrame({

    "Feature": X.columns,

    "Importance": rf_model.feature_importances_

})

feature_importance =
feature_importance.sort_values(
```

```
    by="Importance",
    ascending=True
)

plt.figure(figsize=(9, 6))

plt.barh(
    feature_importance["Feature"],
    feature_importance["Importance"],
    color="teal",
    edgecolor="black"
)

plt.title("Random Forest - Feature Importance")

plt.xlabel("Importance Score")

plt.ylabel("Features")

plt.grid(axis="x", alpha=0.3)

plt.tight_layout()

plt.show()

# Create pollution categories

def classify_pollution(value):

    if value <= 30:

        return "Good"

    elif value <= 60:

        return "Satisfactory"

    elif value <= 90:

        return "Moderate"

    elif value <= 120:

        return "Poor"

    elif value <= 250:

        return "Very Poor"

    else:

        return "Severe"

station_data["Pollution_Level"] = (

station_data["PM2.5"].apply(classify_pollution)
```

```
)

class_data = station_data.dropna(
    subset=["PM2.5"]
).copy()

X_class = class_data[
    [
        "PM10",
        "NO2",
        "SO2",
        "CO",
        "OZONE",
        "NH3",
        "latitude",
        "longitude"
    ]
]
y_class = class_data["Pollution_Level"]

print("\nPOLLUTION CATEGORY COUNTS")

print(y_class.value_counts())

X_train_class, X_test_class, y_train_class, y_test_class = train_test_split(
    X_class,
    y_class,
    test_size=0.2,
    random_state=42
)

class_imputer = SimpleImputer(strategy="median")

X_train_class = class_imputer.fit_transform(X_train_class)

X_test_class = class_imputer.transform(X_test_class)

from sklearn.tree import DecisionTreeClassifier

from sklearn.metrics import (
    accuracy_score,
    classification_report,
    confusion_matrix
)

dt_classifier = DecisionTreeClassifier(
    random_state=42
)

dt_classifier.fit(
    X_train_class,
```

```
    y_train_class
)

dt_class_predictions = dt_classifier.predict(
    X_test_class
)

print("\nDECISION TREE CLASSIFIER RESULTS")

print(
    "Accuracy:",
    accuracy_score(y_test_class, dt_class_predictions)
)

print("\nClassification Report")

print(
    classification_report(
        y_test_class,
        dt_class_predictions,
        zero_division=0
    )
)

from sklearn.ensemble import RandomForestClassifier

rf_classifier = RandomForestClassifier(
    n_estimators=100,
    random_state=42
)

rf_classifier.fit(
    X_train_class,
    y_train_class
)

rf_class_predictions = rf_classifier.predict(
    X_test_class
)

print("\nRANDOM FOREST CLASSIFIER RESULTS")

print(
    "Accuracy:",
    accuracy_score(y_test_class, rf_class_predictions)
)

print("\nClassification Report")

print(
    classification_report(
```

```
        y_test_class,
        rf_class_predictions,
        zero_division=0
    )
)
```

```
from sklearn.preprocessing import StandardScaler

from sklearn.svm import SVC


# Feature Scaling

scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(
    X_train_class
)

X_test_scaled = scaler.transform(
    X_test_class
)


# Train SVM Model

svm_model = SVC()

svm_model.fit(
    X_train_scaled,
    y_train_class
)


svm_predictions = svm_model.predict(
    X_test_scaled
)


print("\nSVM CLASSIFIER RESULTS")

print(
    "Accuracy:",
    accuracy_score(y_test_class, svm_predictions)
)

print("\nClassification Report")

print(
    classification_report(
        y_test_class,
        svm_predictions,
        zero_division=0
    )
```

```
)

classification_results = pd.DataFrame({
    "Model": [
        "Decision Tree",
        "Random Forest",
        "SVM"
    ],
    "Accuracy": [
        accuracy_score(y_test_class, dt_class_predictions),
        accuracy_score(y_test_class, rf_class_predictions),
        accuracy_score(y_test_class, svm_predictions)
    ]
})

print("\nCLASSIFICATION MODEL COMPARISON")
print(classification_results)

plt.figure(figsize=(7, 6))
plt.scatter(
    y_test,
    rf_predictions
)

plt.xlabel("Actual PM2.5")
plt.ylabel("Predicted PM2.5")
plt.title("Actual vs Predicted PM2.5 using Random Forest")
plt.show()

labels = rf_classifier.classes_
cm = confusion_matrix(
    y_test_class,
    rf_class_predictions,
    labels=labels
)

plt.figure(figsize=(9, 7))
sns.heatmap(
```

```
    cm,
    annot=True,
    fmt="d",
    cmap="Blues",
    xticklabels=labels,
    yticklabels=labels
)

plt.title("Random Forest Pollution Classification")

plt.xlabel("Predicted")

plt.ylabel("Actual")

plt.show()



feature_names = [
    "PM10",
    "NO2",
    "SO2",
    "CO",
    "OZONE",
    "NH3",
    "latitude",
    "longitude"
]
```

```
importance = rf_model.feature_importances_



feature_importance = pd.DataFrame({
    "Feature": feature_names,
    "Importance": importance
})

feature_importance = feature_importance.sort_values(
    by="Importance",
    ascending=False
)



print("\nRANDOM FOREST FEATURE IMPORTANCE")

print(feature_importance)



print("\n==============================")

print("AIR POLLUTION INTELLIGENCE PROJECT")
```

```
print("ALL MODELS EXECUTED
SUCCESSFULLY")


print("==============================
======")
```